

AI Tools & Usage

Two things, kept strictly separate: the AI I built *with* — and the AI that runs *inside* the product

Product	AI Study Companion — AI-Powered Learning & Growth Workspace
Live deployment	https://ai-study-companion-three-inky.vercel.app/
Runtime AI	Mercury mercury-2.5 (chat / structured / eval) · Gemini gemini-embedding-2 (768-d embeddings) — via lib/ai/AIService.ts
Build AI	Claude (architecture & build prompts) · OpenCode coding agent · Muse Spark model (implementation)
Date	18 September 2026

How to read this document. Reviewers often conflate “AI-assisted development” with “AI features in the app”. They are different claims with different evidence. **Part A** covers process (what helped me build). **Part B** covers product (what the user interacts with, with guardrails and observability). Nothing in Part A is presented as product capability, and nothing in Part B was “just prompted” — each runtime feature has deterministic guardrails around the model.

Part A — AI used to BUILD this product

I built this prototype solo on a 3–4 day clock. I used AI as a force-multiplier for planning, implementation, debugging and docs — but accounts, keys, deploys, smoke tests and the demo video were done by hand, and every verification claim below comes from commands I actually ran.

Tool	What I used it for (concrete)
Claude architecture + prompt generation	Drafted <code>extra-docs/docs/architecture.md</code> — the single source of truth (schema, RAG flow, security model, event system, MUST/SHOULD/FUTURE scope) — and the ordered build prompts in <code>build-prompts.md</code> (phases 02–19). This front-loaded the hard thinking so implementation could move phase-by-phase without re-deciding the design daily.
OpenCode coding agent (all phases)	Implemented every phase from those prompts: scaffolding, migrations 001–014, auth, services, RAG, Tutor/Quiz/Assessment/Mastery/Growth/Recommendations, analytics, admin (11 views), observability, tests + eval harness, deployment prep, docs. Also the mid-build fixes that mattered: closing the <code>EMBEDDING/CONCEPT_EXTRACTION</code> logging gaps, fixing the circular-type build error in retrieval fixtures, Gemini 768-d migration + purge/reindex, material-dedupe + inline fallback so Vercel never strands uploads in <code>QUEUED</code> . Wrote per-phase logs into <code>development-prompts/</code> .
Muse Spark model behind later sessions	The LLM powering the OpenCode agent for the later phases (observability, eval suite, deployment, sidebar rework) and the continuation tasks: Home Dashboard PRD alignment (<code>dashboard.service.ts</code>), Admin System Health (<code>/admin/health</code>), Tutor conversation-summary continuity (migration 005 + rolling summary), evaluation run-tracking (<code>evaluation.service.ts</code> + run-over-run comparison), flashcards / practice / misconceptions / knowledge-graph extras, and this documentation set. Issues: https://github.com/anomalyco/opencode .

How I worked with the agent (my actual loop)

1. **One prompt at a time, in build order.** I pasted `architecture.md` once as context, then fed one phase prompt, waited for green acceptance checks, and only then moved on.
2. **Acceptance checks were real commands** — `npm test` (314 passing), `npm run eval` (18/18 fixtures), `npm run build / lint / typecheck`, plus live Supabase queries. Outputs were recorded into phase logs and `evaluation.md`, not hypothesised.
3. **/compact between phases.** Each phase ended with a context compaction; I saved the exact prompt + result + deviations into `development-prompts/NN-name.md` (template at the bottom of `build-prompts.md`). That folder is the raw history; the “AI Prompts” PDF is its cleaned, organised companion.
4. **I stayed the reviewer.** I kept, cut or rewrote agent output — e.g. I rejected an over-generic recommendation prompt and replaced it with one that bans “keep studying” and forces named concepts; I simplified mastery to the explainable 0.7/0.3 formula rather than the agent’s heavier first draft.

What AI did NOT do (so expectations stay honest): it did not create my Supabase / Vercel / Inngest accounts, did not set production env vars, did not run the live-URL smoke test (signup → space/project → PDF → READY → Tutor → quiz → mastery/growth/recommendations — manual), and did not record the demo video. Any gap between “passes locally” and “works live” was closed by me, by hand, on the deployed URL.

Effort picture (approximate, honest)

Workstream	AI role	Human role
Architecture & phasing	Drafted, I edited heavily	Final decisions + scope cuts (MUST/SHOULD/FUTURE)
Routine code (CRUD, forms, admin tables)	Generated ~70–80%	Reviewed every route; fixed ownership scoping
RAG / Tutor / grading (core IP)	Scaffolded	Designed thresholds, validators, insufficient-evidence path, injection tests
Debugging (build, RLS, Inngest, dims)	Diagnosed with me	Reproduced, chose fixes, re-ran suites
Docs & submission PDFs	Drafted from repo evidence	Corrected stale stack claims, verified counts

Part B — AI used BY the final product

All runtime AI flows through one thin abstraction — `lib/ai/AIService.ts` — so providers can change without touching features. Current split:

 **Chat / structured / evaluation** → **Mercury** mercury-2.5 (<https://api.inceptionlabs.ai/v1>) — JSON mode + per-feature validators; reasoning-model shaping (token headroom, temp clamp [0.5,1], reasoning_effort=low for structured tasks); 90 s timeout.

 **Embeddings** → **Google Gemini** gemini-embedding-2, 768 dims (`chunks.embedding VECTOR(768)`, migration 012) — same model + config for documents (`title | text`) and queries (`task: search result | query`); key server-only; 10-min query cache.

Feature (operation)	AI role	Human / deterministic guardrails
Grounded Tutor TUTOR	Answers from retrieved chunks → {answer, confidence, grounded, citations, followUp}	Project-scoped retrieval (threshold 0.25, top-K 5); insufficient evidence → fixed response, <i>no LLM call</i> ; evidence in <retrieved_evidence> as untrusted data; 6-msg window + rolling summary are context-only; citations filtered to real chunk IDs; every call logged.
Conversation summary CONV_SUMMARY	Compresses older turns → rolling summary on <code>conversations.summary</code>	Structured {summary, keyTopics}, validated + truncated ≤1200 chars; refresh ~every 6 msgs past 8; best-effort, never blocks answer.
Quiz generation QUIZ_GEN	Writes N questions for selected concepts/difficulties/types	Concept pick is backend weighted scoring (never LLM); output validated (4 options, correct ∈ options), retry once; duplicate-question + 2-min guards.
Open-ended grading OPEN_EVAL	Grades free text → {score, understanding, strengths, missingConcepts, reasoning, feedback}	MCQ graded deterministically (no LLM); open-ended validated; failure raises, never persists a guess; score ≥ 60 ⇒ correct.
Concept extraction CONCEPT_EXT	Extracts {name, description}[] (≤8) from uploads	Non-fatal — failure yields [] so material never sticks in PROCESSING; linked via <code>source_material_id</code> .
Sub-concepts · Flashcards SUB / FLASH	Expand a concept; build weak-concept decks	Rate-limited (10/min, 5/min); validated before persist; typed errors.
Recommendations RECOMMEND	Produces {title, action_items[2-5]} from weak concepts + mistakes + goal + activity	Prompt bans generic advice; validator enforces named concepts; skipped when nothing weak (<60 or REQUIRES_ATTENTION).
Practice & misconceptions	Sectioned practice + MCQ; repeated-mistake patterning	Same validators + rate limits; misconception bump/merge is backend logic.
Embeddings EMBEDDING	Chunk + query vectors for pgvector cosine	<code>match_chunks</code> filtered by <code>project_id</code> ; dim gate 768; one <code>ai_operations</code> row per call incl. failures with <code>request_id</code> .

Deterministic — never LLM-decided: mastery math ($\text{prev} \times 0.7 + \text{evidence} \times 0.3$) · growth bands (± 5) · eval run comparison · ownership checks (`WHERE id AND user_id + RLS`) · all analytics aggregations · MCQ grading · answer gating.

Cost, latency & debuggability

Every call writes `ai_operations(feature, model, request_id, latency_ms, success, tokens, estimated_cost, error)`, surfaced in `/admin/ai-usage` and `project/global analytics`. Cost is controlled structurally: embeddings only for real work (processing batches + uncached queries — never on page open, never re-indexing READY material), per-user rate windows (tutor 20/min, quiz-gen 5/min, quiz-submit 60/min... → 429 + Retry-After), and conversation prompts bounded by the 6-message window + summary. An on-call question like *“why was that Tutor answer slow / which model / why poor retrieval / which workflow failed / what did it cost?”* is answerable from one table join. (Honest note: provider usage objects aren't yet threaded into every row, so token/cost cells can be null — calls, latency and errors are complete; cost capture is queued next.)

Safety posture (runtime)

- **Isolation:** retrieval, conversations, chunks, quizzes and recommendations are all project-ownership-gated; cross-project reads 404.
- **Injection:** Tutor / practice / concept prompts delimit retrieved text as untrusted; remaining generators are queued for the same treatment.
- **Validation:** no raw LLM text is persisted — every feature has a server-side schema check; hallucinations (e.g. fake chunk IDs) are dropped and groundedness downgraded.
- **Secrets:** Mercury + Gemini keys are server-only env vars; the browser never sees them.

Evaluation linkage: runtime quality is checked by 18 curated fixtures (grounded / unsupported / multi-concept / citation / injection; retrieval relevance; grading quality; recommendation actionability) via `npm run eval`, with run-over-run IMPROVED / REGRESSED / UNCHANGED / BASELINE tracking in `/admin/ai-evaluation` — so a prompt, model or retrieval change that regresses behaviour is caught, not silently shipped.

Bottom line: Part A made a 3–4 day prototype feasible; Part B is what's actually being graded — a small set of well-guarded model calls wrapped in deterministic learning logic, full observability, and an eval harness. The model could be swapped tomorrow; the architecture wouldn't need to change.